



AIOps Intelligence
Engine

SOLUTION DESIGN · HLD + LLD

AIOps Intelligence Engine

AI-powered anomaly detection, alert correlation, causal topology,
predictive forecasting & LLM root-cause analysis for Kubernetes — air-
gapped.

Version 1.0 · **Status** Production-Readiness Baseline

Date 2026-06-02

AIOps Intelligence Engine — Solution Design Document (HLD + LLD)

AI-powered anomaly detection, alert correlation, causal topology, predictive forecasting, and LLM root-cause analysis for Kubernetes — **air-gapped**, with a local Ollama LLM.

Document	Solution Design — HLD & LLD
Version	1.0
Status	Production-readiness baseline
Date	2026-06-02
Scope	Backend (FastAPI), ML pipeline, data model, deployment, prod hardening
Code of record	/opt/aiops-engine/backend , /opt/aiops-engine/helm

How to read the diagrams: all diagrams are Mermaid and render natively in GitHub, GitLab, and VS Code (Markdown Preview Mermaid Support). No external tooling required — they live in source control with the code.

Table of Contents

- Executive Summary
- Requirements & Design Drivers
- High-Level Design (HLD) - 3.1 System Context - 3.2 Logical Architecture (Block Diagram) - 3.3 End-to-End Data Flow - 3.4 Deployment Architecture (Kubernetes)
- Low-Level Design (LLD) - 4.1 Component Model - 4.2 Concurrency & Runtime Model - 4.3 Data Model (ERD) - 4.4 Sequence Diagrams - 4.5 Algorithm Detail - 4.6 API Surface - 4.7 State Machines
- Production Readiness - 5.1 Scalability & HA - 5.2 Reliability & Failure Modes - 5.3 Security - 5.4 Self-Observability - 5.5 Capacity & Performance - 5.6 CI/CD & Release - 5.7 Disaster Recovery
- Known Gaps & Hardening Roadmap
- Appendix: Config Reference

1. Executive Summary

The AIOps Intelligence Engine is a single-deployable **decision layer** that sits on top of an existing Prometheus/Alertmanager/OTEL stack and converts raw signals into **ranked, explained, actionable incidents**. It does five things the underlying monitoring stack does not:

- ML anomaly detection** — an ensemble (Isolation Forest + Z-Score + EWMA) over scraped metric series.

2. **Alert correlation** — Union-Find clustering with temporal + label + semantic (TF-IDF) scoring to collapse alert storms into incidents.
3. **Causal topology** — a learned directed dependency graph used to pick the *causal root* of an incident.
4. **Predictive forecasting** — Holt damped-trend projection that raises *predicted* incidents before a threshold breach.
5. **LLM RCA** — structured root-cause analysis and remediation via a local, air-gapped Ollama model.

Design north-star: runs **fully offline** (no SaaS, no egress), is **K8s-native** (Helm), and is **portable** (SQLite default, Postgres for scale).

2. Requirements & Design Drivers

Driver	Implication on design
Air-gapped / data-sovereign	Local LLM (Ollama), no external APIs, no telemetry egress.
Bolt-on, not rip-and-replace	Consumes Prometheus/Alertmanager/OTEL; never owns the source of truth.
Low operational footprint	Single backend image; SQLite default; one Helm chart.
Real-time ops UX	Server-Sent Events (SSE) push; same-origin SPA.
Explainability	Every incident carries a causal root + LLM rationale + confidence.
CPU-only inference reality	llama3 on CPU ≈ 75 s/call $\rightarrow$ RCA & inference are deduped/cached aggressively.

Non-functional targets (baseline)

NFR	Target (single-node baseline)
Alert ingest throughput	≥ 200 alerts/s burst (webhook)
Detection sweep latency	$\leq$ <code>ANOMALY_DETECTION_INTERVAL</code> (60 s) per cycle
Correlation freshness	≤ 30 s after alert lands
RCA latency (CPU Ollama)	$\sim 75\text{--}120$ s (async, non-blocking; deduped)
UI event push latency	< 1 s via SSE
Availability (HA mode)	99.9% (Postgres + N replicas)

3. High-Level Design (HLD)

3.1 System Context

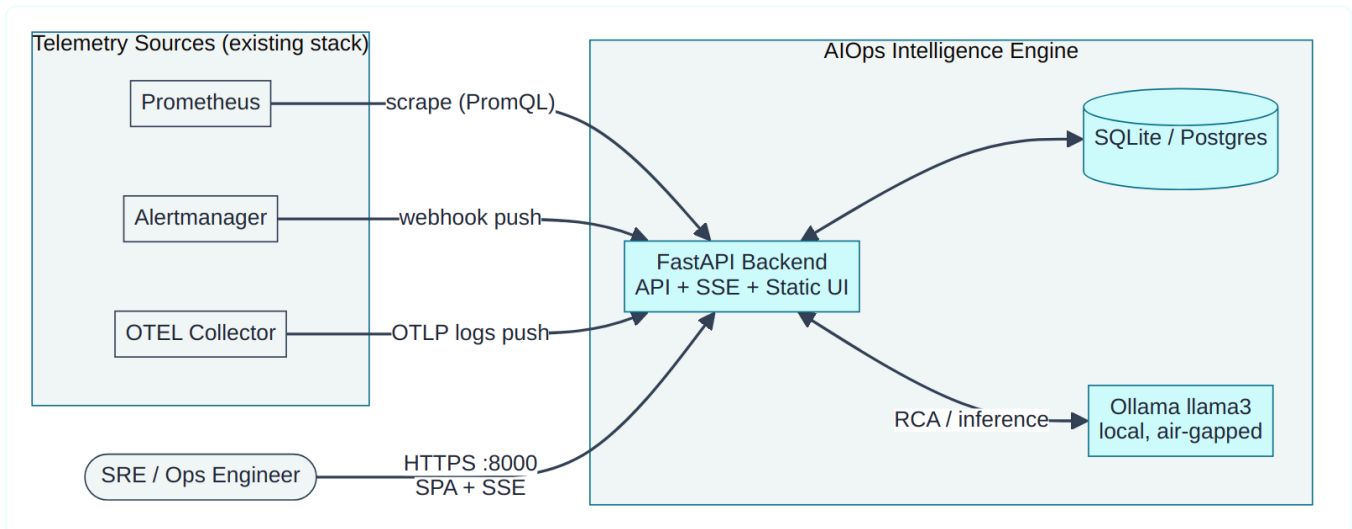


Figure 1

Boundary note: the Engine is a *consumer* of the monitoring stack. Prometheus remains the metric store of record; the Engine keeps only derived state (alerts, anomalies, correlations, topology, RCA).

3.2 Logical Architecture (Block Diagram)

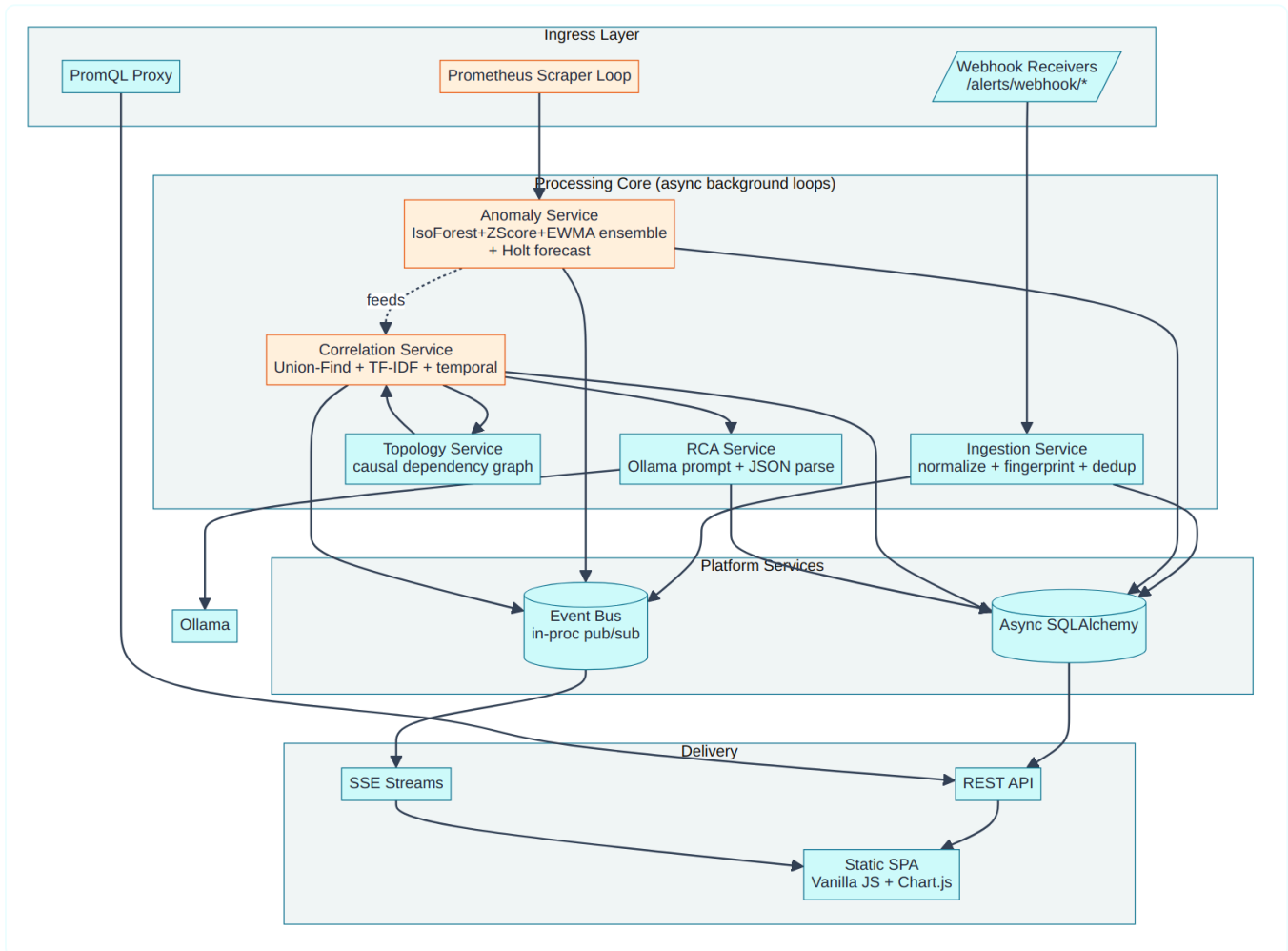


Figure 2

Three always-on background loops (started in `app/main.py` lifespan):

Loop	Source	Cadence	Owner
<code>prometheus-scraper</code>	<code>IngestionService.start_prometheus_scraper</code>	scrape interval	metrics
<code>anomaly-detector</code>	<code>AnomalyService.start_detection_loop</code>	<code>ANOMALY_DETECTION_INTERVAL</code>	detection + forecast
<code>correlator</code>	<code>AlertCorrelationService.start_correlation_loop</code>	windowed	correlation + topology + inference

3.3 End-to-End Data Flow

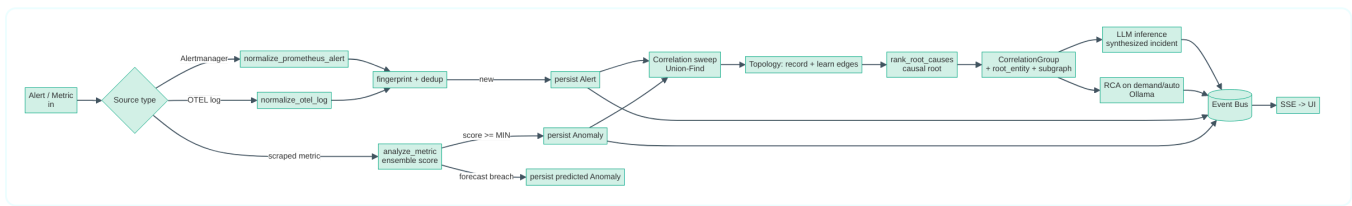


Figure 3

3.4 Deployment Architecture (Kubernetes)

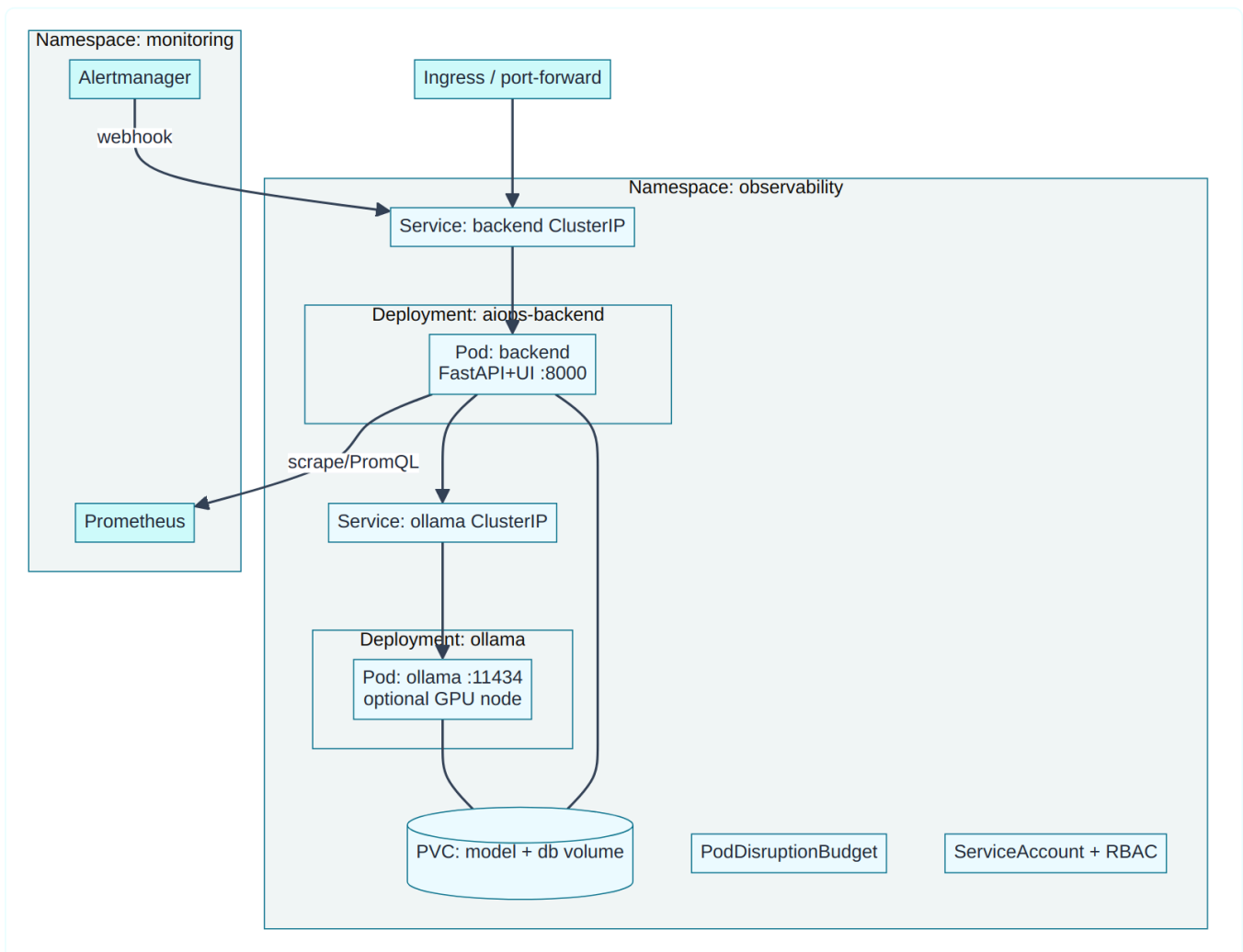


Figure 4

Current chart (helm/templates): deployment.yaml, services.yaml, frontend-html.yaml, _helpers.tpl.
Production additions are listed in §6 (HPA, NetworkPolicy, ServiceMonitor, externalised Postgres, secrets).

Local dev / single-box runs the same backend as a **systemd service** (aiops-engine.service) on :8000 — see docs /memory aiops-local-run.

4. Low-Level Design (LLD)

4.1 Component Model

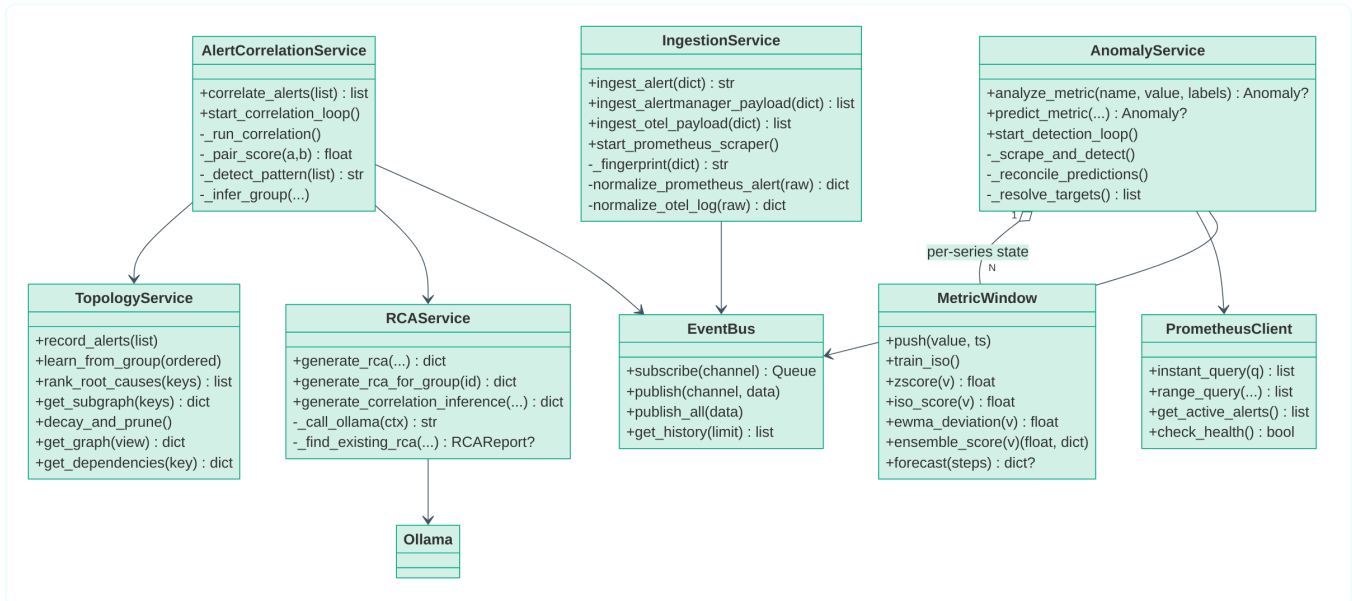


Figure 5

4.2 Concurrency & Runtime Model

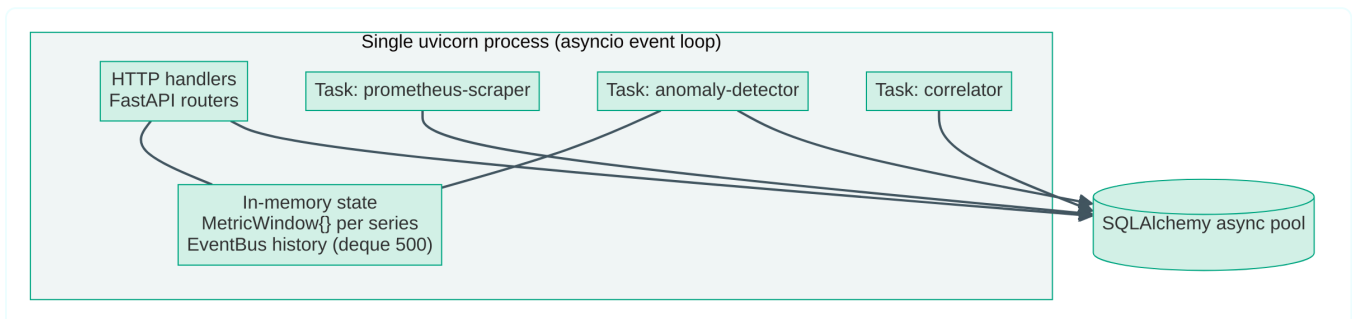
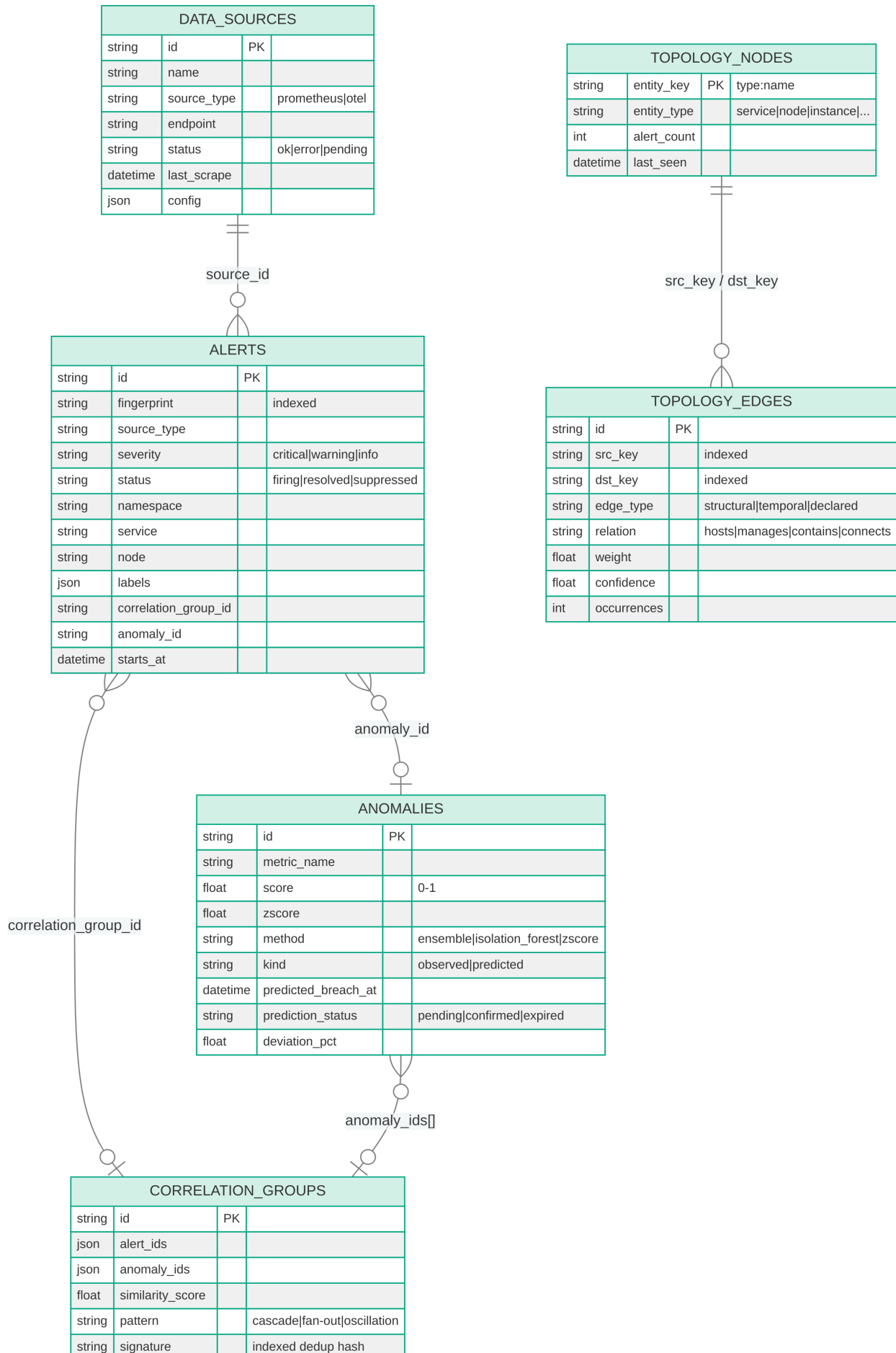


Figure 6

Critical design facts (impact prod scaling — see §5.1):

- The three loops are `asyncio.create_task`s in the **same process** as the API; they share the event loop and the SQLAlchemy async engine.
- MetricWindow** per-series rolling buffers live in **process memory** (Isolation Forest models, EWMA state). This is **node-local, non-shared state** → naïvely running N replicas would give each replica a *partial* view and *duplicate* detection. **Mitigation in §5.1.**
- EventBus** is an **in-process** pub/sub with a bounded history deque → SSE clients must be served by the replica that holds the relevant events (sticky), or events must move to an external bus (Redis/NATS) for multi-replica.
- Graceful shutdown cancels all loop tasks in `lifespan` teardown (`SIGINT` honored by the systemd unit / container).

4.3 Data Model (ERD)



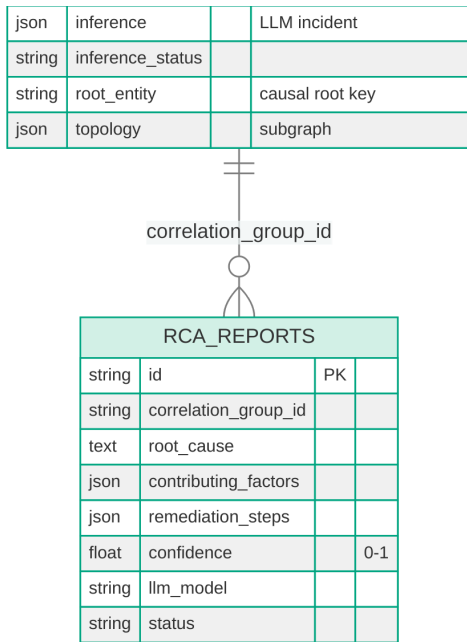


Figure 7

4.4 Sequence Diagrams

4.4.1 Alert → Incident → RCA (the golden path)

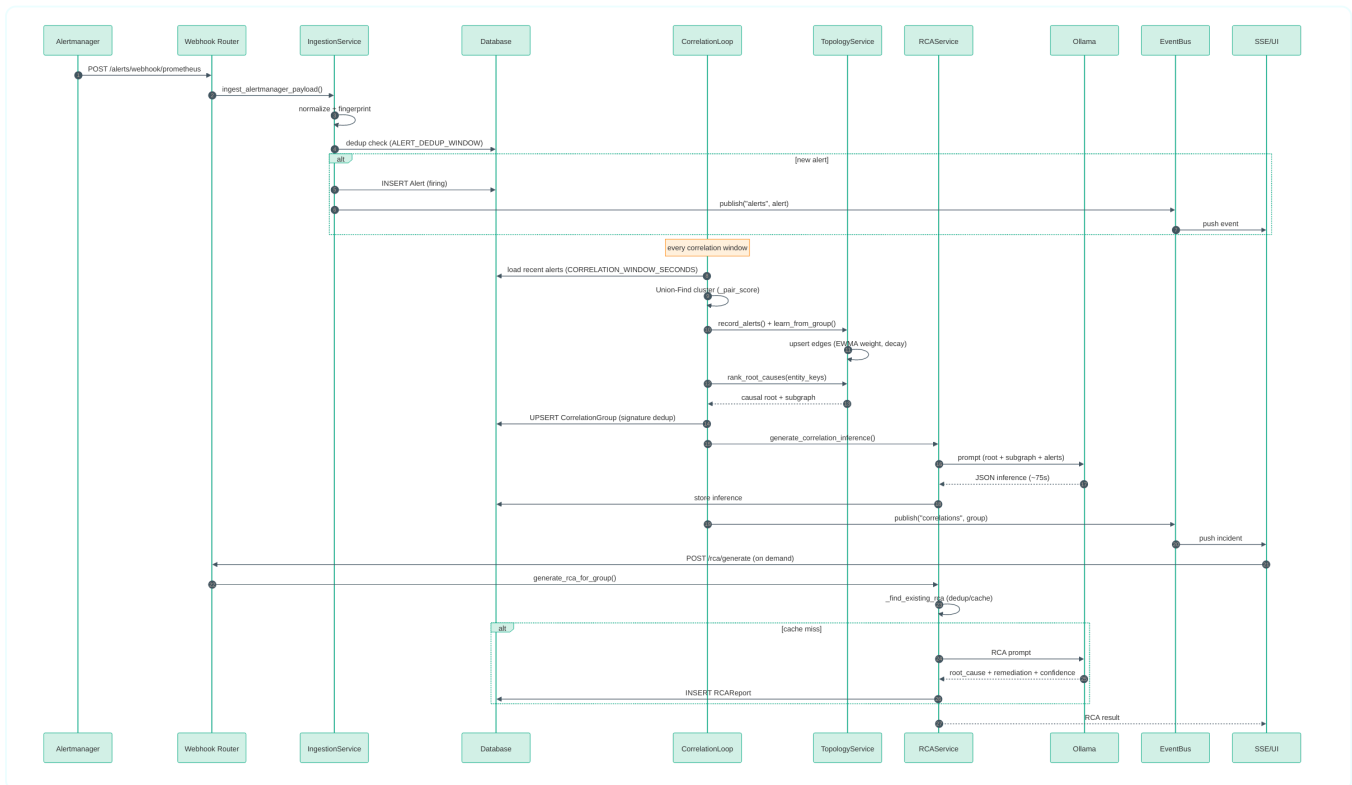


Figure 8

4.4.2 Predictive Forecasting (preventive ops)

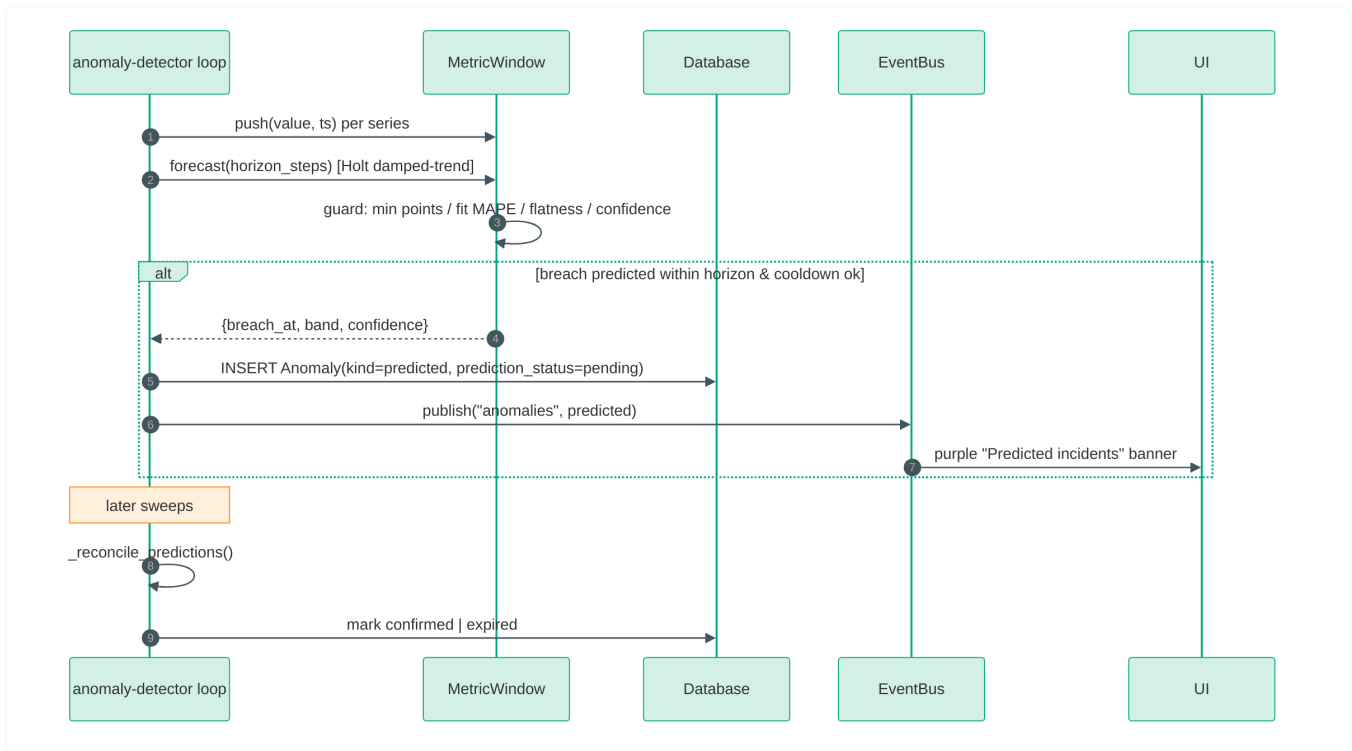


Figure 9

4.4.3 SSE Real-Time Delivery

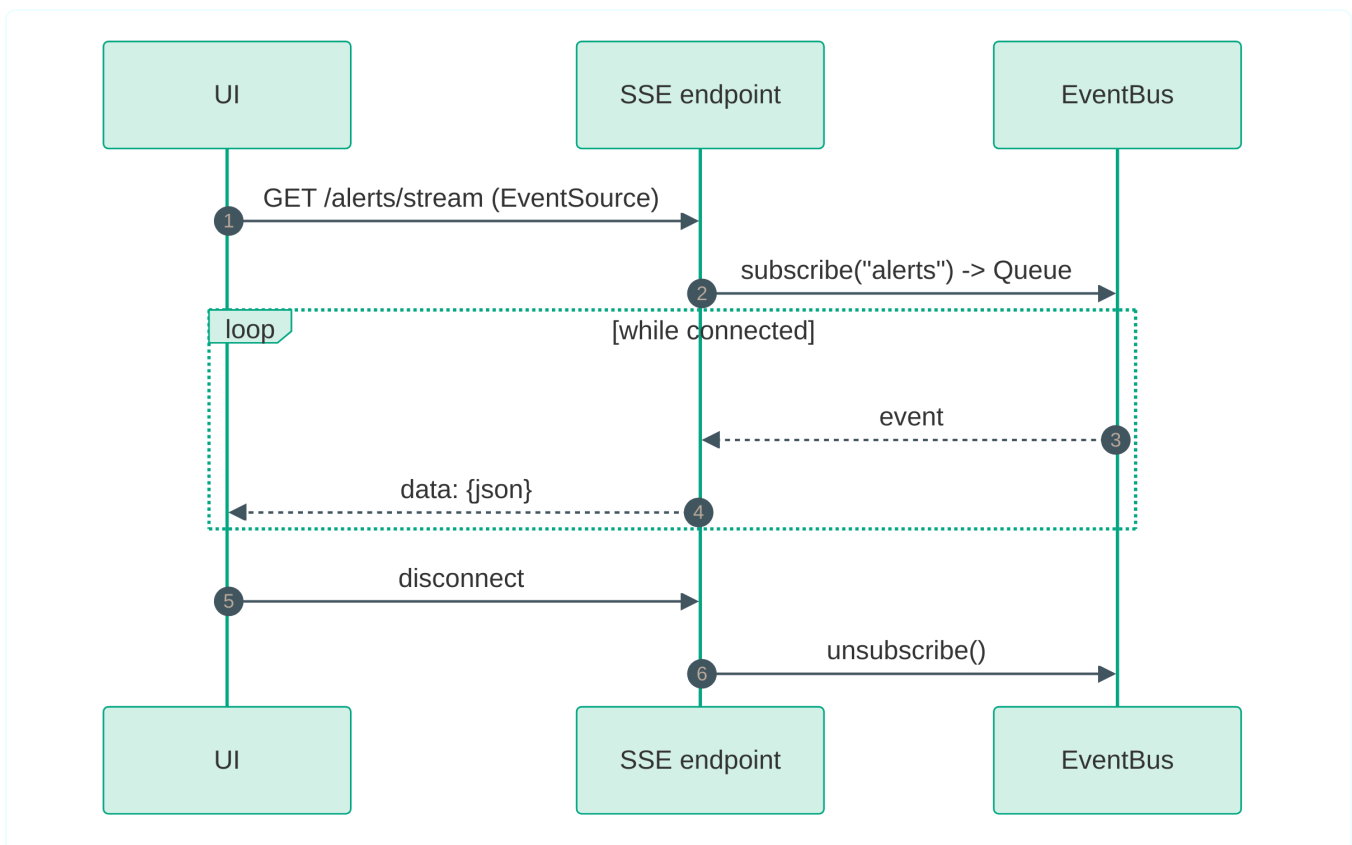


Figure 10

4.5 Algorithm Detail

Anomaly ensemble (`MetricWindow.ensemble_score`):

```
ensemble = 0.50 * isolation_forest_score
          + 0.30 * normalized_zscore
          + 0.20 * ewma_deviation
emit Anomaly iff ensemble >= MIN_ANOMALY_SCORE (0.6)
```

- Isolation Forest retrains per series on a rolling window when `needs_retrain()` (contamination = `ISOLATION_FOREST_CONTAMINATION`).
- Z-Score threshold `ZSCORE_THRESHOLD` (3σ); EWMA `alpha=0.1`.

Correlation pair score (`_pair_score`): weighted sum of label similarity, namespace match, temporal proximity (within `CORRELATION_WINDOW_SECONDS`), and TF-IDF cosine over alert names → Union-Find clustering → pattern label (`cascade` | `fan-out` | `oscillation`). Groups deduped by a **signature hash** of sorted member IDs so the same cluster is not re-inferred each sweep.

Causal topology (`rank_root_causes`): directed edges carry `weight` (EWMA strengthen + multiplicative `TOPOLOGY_EDGE_DECAY` per sweep, pruned below `TOPOLOGY_MIN_EDGE_WEIGHT`, TTL `TOPOLOGY_EDGE_TTL_SECONDS`). Root = entity with max net (out – in) influence, **cycle-safe**. Edge types: `structural` | `temporal` | `declared`.

Forecast (`MetricWindow.forecast`): Holt damped-trend with prediction intervals; predicts z-band breach + ETA. Guards: cold-start (`FORECAST_MIN_POINTS`), flat/ degenerate series, poor in-sample fit (`MAPE > FORECAST_MAX_FIT_ERROR`), low confidence (`< FORECAST_MIN_CONFIDENCE`), per-series cooldown (`FORECAST_COOLDOWN_SECONDS`).

4.6 API Surface

Method	Path	Purpose	Notes
POST	/api/v1/alerts/webhook/prometheus	Alertmanager receiver	idempotent (fingerprint)
POST	/api/v1/alerts/webhook/otel	OTEL log receiver	
GET	/api/v1/alerts	list/filter alerts	
GET	/api/v1/alerts/stream	SSE alert stream	EventSource
GET	/api/v1/anomalies	list anomalies	
GET	/api/v1/anomalies/predicted	predicted incidents	forecasting
GET	/api/v1/anomalies/stream	SSE anomaly stream	
GET	/api/v1/correlations	correlation groups	
GET	/api/v1/topology?view=infra\ all	dependency graph	ITOM infra view default
GET	/api/v1/topology/dependencies?entity=	upstream/downstream	drill-down
POST	/api/v1/topology/rebuild	re-derive graph	
POST	/api/v1/topology/edges	declare edge	
POST	/api/v1/rca/generate	trigger RCA	deduped/cached
GET	/api/v1/rca	list RCA reports	
GET/POST	/api/v1/sources	manage data sources	DB-driven
GET	/api/v1/metrics/query , /query_range	PromQL proxy	
GET	/api/v1/health/full	dependency health	Prom/Ollama/AM

4.7 State Machines

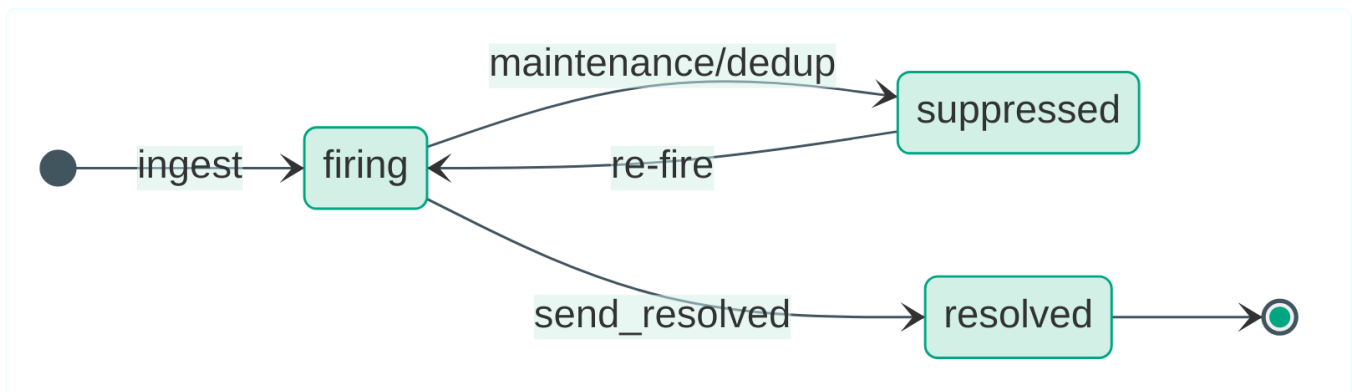


Figure 11

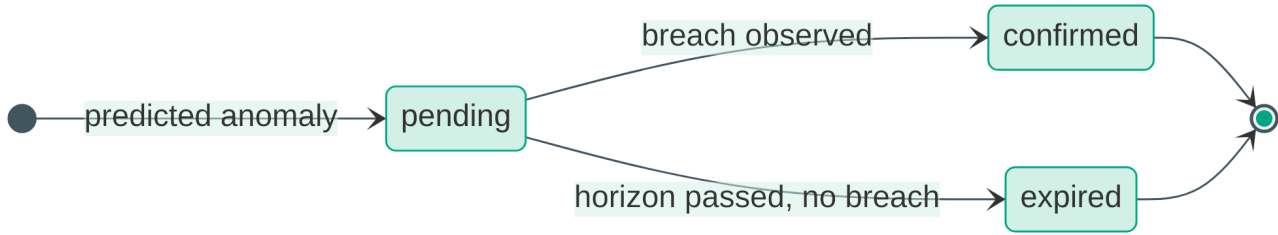


Figure 12

5. Production Readiness

5.1 Scalability & HA

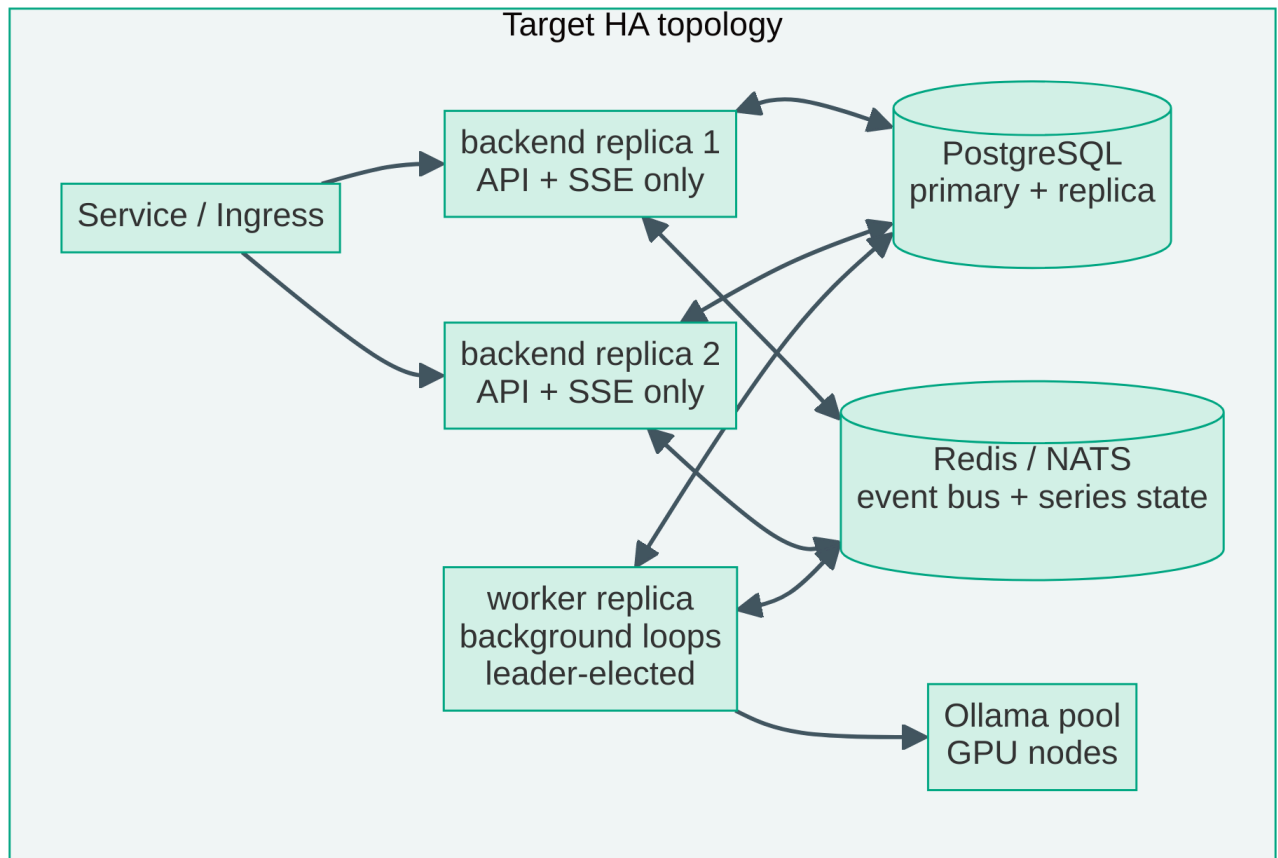


Figure 13

Concern	Current	Production path
DB single-writer	SQLite (one writer)	Postgres (<code>postgresql+asyncpg</code>), already a documented swap.
In-memory <code>MetricWindow</code> per series	node-local	Split roles: run background loops in a single leader-elected worker Deployment; API replicas are stateless. Or externalise series state to Redis.
In-proc EventBus	not shared across pods	Move to Redis pub/sub or NATS so any API replica can serve SSE.
RCA throughput	1 CPU Ollama, ~75s	GPU node pool + a small queue; RCA already deduped/cached so demand is bounded.
Horizontal API scale	works once state externalised	HPA on CPU/RPS for API replicas.

Key takeaway: the engine is currently a **correct single-node system**. The path to HA is **role separation** (stateless API replicas + one worker that owns the loops) + **externalising the two pieces of in-memory state** (series buffers, event bus). No algorithm changes required.

5.2 Reliability & Failure Modes

Failure	Behaviour today	Hardening
Prometheus unreachable	scraper logs + retries next cycle	backoff + circuit breaker; surface in <code>/health/full</code> .
Ollama down / slow	RCA/inference fail or block on timeout (<code>OLLAMA_TIMEOUT=120</code>)	bounded queue + <code>inference_status=failed</code> already exists; add retry w/ jitter.
Alert storm	dedup + correlation collapse	rate-limit ingest; cap <code>MAX_ALERTS_MEMORY</code> .
DB lock (SQLite)	writer contention under load	Postgres for concurrency.
Pod restart	loops restart; in-mem series cold	warm from DB-backed history; accept short cold-start (guards already skip cold series).
Loop crash	task dies silently	add supervised restart + health gauge per loop (gap, §6).

5.3 Security

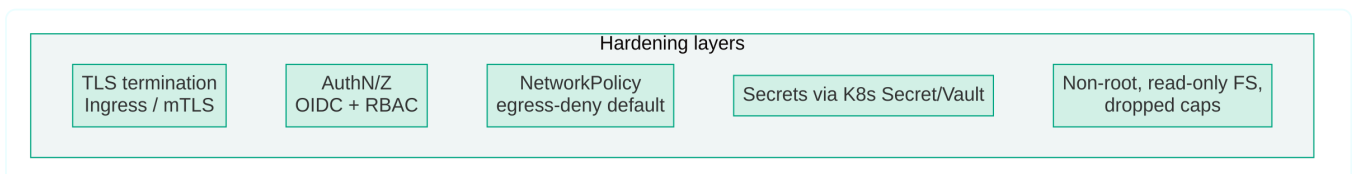


Figure 14

Area	Current	Action
AuthN/AuthZ	none on API	Add OIDC/JWT middleware + role model (read-only vs operator). Priority.
CORS	allow_origins=["*"]	restrict to UI origin in prod.
Secrets	SECRET_KEY="change-me" , creds in .env	K8s Secret / Vault; rotate; never bake into image.
TLS	VERIFY_TLS toggle for self-signed	mount internal CA bundle; set VERIFY_TLS=true .
Egress	air-gapped by design	enforce with NetworkPolicy egress-deny (only Prom/AM/Ollama).
Container	n/a	run as non-root, read-only rootfs, drop caps, seccomp.
Input	webhook unauthenticated	shared-secret/bearer on webhook endpoints.
Audit	none	structured audit log for RCA/action endpoints.

5.4 Self-Observability

The engine must be observable too (dogfooding):

- **Metrics:** expose `/metrics` (Prometheus) — ingest rate, detection latency, loop heartbeat gauges, Ollama call duration/queue depth, DB pool stats, SSE client count.
- **Tracing:** OTEL instrument FastAPI + httpx calls (Prom/Ollama).
- **Logs:** already structured (`%(asctime)s | %(levelname)s | %(name)s`); ship to the same OTEL pipeline.
- **ServiceMonitor** + dashboards/alerts for the loops (e.g. "anomaly loop stale > 3 cycles").

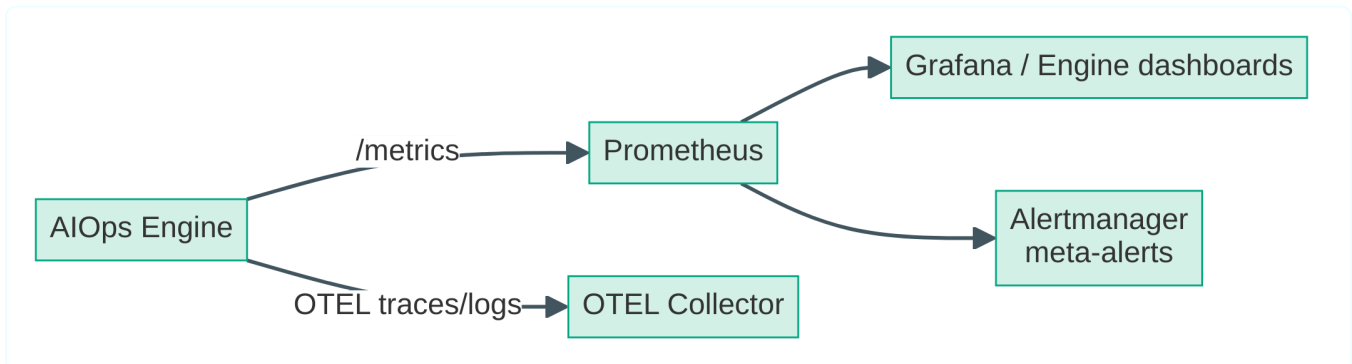


Figure 15

5.5 Capacity & Performance

Dimension	Sizing guidance
Backend pod	1–2 vCPU / 1–2 GiB (API+loops, sklearn in-proc)
Ollama (CPU)	4–8 vCPU / 8 GiB; ~75 s/llama3 call
Ollama (GPU)	1× NVIDIA, ~3–8 s/call → enables auto-RCA at scale
Postgres	2 vCPU / 4 GiB + PVC; index <code>fingerprint</code> , <code>signature</code> , <code>src/dst_key</code> , <code>kind</code>
Series memory	~O(active_series × window_points) floats; cap with retrain window
Retention	roll off resolved alerts / expired predictions (cron); keep RCA long-term

5.6 CI/CD & Release

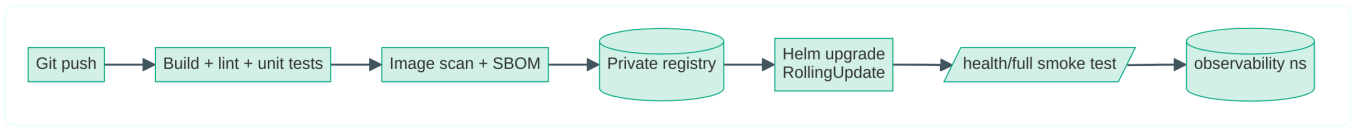


Figure 16

- Versioned image + `helm upgrade` with `RollingUpdate` + PDB (already present).
- DB migrations: introduce **Alembic** before Postgres cutover (currently `create_all` on boot — fine for SQLite, unsafe for evolving Postgres schema).
- Smoke gate on `/api/v1/health/full`.

5.7 Disaster Recovery

Asset	Strategy
Derived state DB	Postgres PITR / scheduled <code>pg_dump</code> ; SQLite → volume snapshot.
Ollama model	bake into image or pre-pull to PVC (air-gapped registry).
Config	Helm values + Secrets in Git/Vault (GitOps).
RPO/RTO	RPO ≤ 24h (state is <i>derived</i> — Prometheus is source of truth, so loss is tolerable); RTO ≤ 15 min via <code>helm install</code> + re-scrape.

DR advantage: because the engine holds only *derived* state, a total loss is recoverable by redeploying and letting the loops rebuild from live telemetry (`/topology/rebuild` re-derives the graph from recent alerts).

6. Known Gaps & Hardening Roadmap

Prioritised, mapped to the feature analysis from the prior review.

#	Item	Type	Priority
1	AuthN/AuthZ on API + webhook secret	Security	P0
2	Externalise series state + event bus (Redis) → multi-replica	Scale	P0
3	Leader-elected worker / API role split	Scale/HA	P0
4	Postgres + Alembic migrations	Data	P0
5	Loop supervision + heartbeat gauges + <code>/metrics</code>	Reliability/Obs	P1
6	NetworkPolicy, non-root, read-only FS, secrets to Vault	Security	P1
7	Incident management layer (group → incident, state, timeline)	Feature	P1
8	Notification routing (Slack/Teams/SMTP/webhook)	Feature	P1
9	SLO / error-budget tracking	Feature	P1
10	Alert lifecycle: flapping, maintenance windows, suppression	Feature	P2
11	Guarded auto-remediation / runbooks	Feature	P2
12	GPU Ollama pool for auto-RCA at scale	Perf	P2

Items 7–11 are the "operational workflow" gap identified in the leading-product feature comparison: strong detection/diagnosis exists; the **act-on-it** layer is the highest-leverage build-out.

7. Appendix: Config Reference

All tunables are env vars (`app/core/config.py`), overridable via `.env` (local) or `helm values → backend.env` (cluster).

Group	Keys
App	<code>APP_NAME</code> , <code>DEBUG</code> , <code>SECRET_KEY</code>
DB	<code>DATABASE_URL</code>
Prometheus/AM	<code>PROMETHEUS_URL</code> , <code>PROMETHEUS_SCRAPE_INTERVAL</code> , <code>ALERTMANAGER_URL</code> , <code>VERIFY_TLS</code>
OTEL	<code>OTEL_GRPC_ENDPOINT</code> , <code>OTEL_HTTP_ENDPOINT</code>
Ollama	<code>OLLAMA_BASE_URL</code> , <code>OLLAMA_MODEL</code> , <code>OLLAMA_TIMEOUT</code>
Detection	<code>ANOMALY_DETECTION_INTERVAL</code> , <code>CORRELATION_WINDOW_SECONDS</code> , <code>ZSCORE_THRESHOLD</code> , <code>ISOLATION_FOREST_CONTAMINATION</code> , <code>MIN_ANOMALY_SCORE</code>
Forecast	<code>FORECAST_ENABLED</code> , <code>FORECAST_HORIZON_SECONDS</code> , <code>FORECAST_MIN_POINTS</code> , <code>FORECAST_MIN_CONFIDENCE</code> , <code>FORECAST_MAX_FIT_ERROR</code> , <code>FORECAST_COOLDOWN_SECONDS</code> , <code>FORECAST_STEP_SECONDS</code>
Topology	<code>TOPOLOGY_ENABLED</code> , <code>TOPOLOGY_EDGE_DECAY</code> , <code>TOPOLOGY_MIN_EDGE_WEIGHT</code> , <code>TOPOLOGY_EDGE_TTL_SECONDS</code> , <code>TOPOLOGY_MAX_NODES</code>
Alerting	<code>ALERT_DEDUP_WINDOW</code> , <code>MAX_ALERTS_MEMORY</code> , <code>CORS_ORIGINS</code>

Generated against code of record at `/opt/aiops-engine` on 2026-06-02. Diagrams are Mermaid; render in any GitHub/GitLab/VS Code Markdown view.